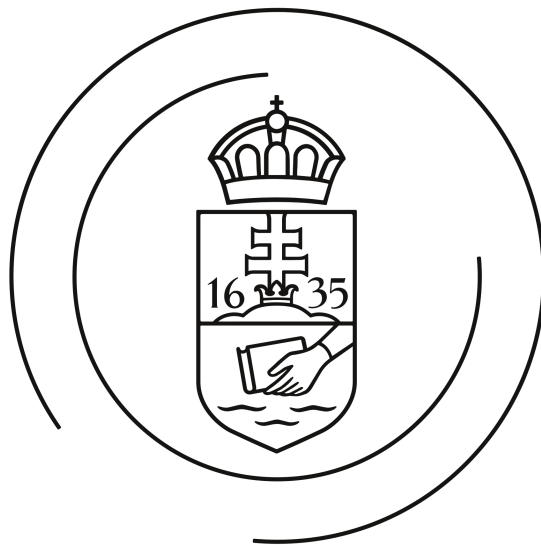


7. számú mérés

Sejtautomaták

Nováki Lóránt



ELTE

Jegyzőkönyv adatok

Neptun kód: TK3EJ9

Leadás ideje: 2025. május 16.

Évfolyam: 3. évfolyam, 6. félév

1. A mérés célja

A mérés célja a sejtautomatákkal való megismerkedés, illetve egy másik hasonló szimuláció, a homokdomb modellel való megismerkedés is része a mérésnek. Ezen két módszer segítségével egyszerű szabályokkal bíró komplex rendszereket lehet szimulálni, amikhez hasonlatosak például az egysejtes élőlények mozgása (sejtautomaták), vagy az erdőtüzek kialakulása (homokdomb modell).

2. A kísérleti elrendezés

A szimulációt C++ program segítségével végeztem el, ami előre meg volt írva nagyrészt. Apró változtatásokat eszközöltem csak rajta, melyekkel kényelmesen tudtam futtatni a notebookból (A lefordított .exe fájlt) a lejjebb látható rövid hívással, amiben a futatáshoz használt paraméterek is láthatóak (a futtatást Python 3.11.8-ban végeztem az ábrák elkészítése és kényelmesség miatt):

```
1 import subprocess
2
3 linear = "n" # y/n is the pendulum linear?
4 L = 9.8 # length of pendulum
5 q = 0.2 # damping coefficient
6 Omega = 0.66666 # frequency of driving force
7 F_D = 1.2 # amplitude of driving force
8 theta = 0.2 # initial angle
9 omega = 0 # initial angular velocity
10 tMax = 100*2 # maximum integration time
11 SimulationMethod = "default" # Euler, EulerCromer, RK4, adaptiveRK4(default)
12
13 fileName = "chaotic.txt" # output file name (string)
14
15 arguments = ["simulator.exe", linear, str(L), str(q), str(Omega), str(F_D), str(theta), str(omega), str(tMax),
16             fileName, SimulationMethod]
17
18 subprocess.run(arguments)
```

C++ szimulációt futtató Python kód

A módosított, ezzel a kóddal futtatható, kismértékben megváltoztatott C++ kód a 6. fejezetben található. A szimulációt asztali számítógépen végeztem Visual Studio Codeban, 12th Gen Intel(R) Core(TM) i3-12100F-as 3.30 GHz-es processzorral.

3. A mérés elve

A méréshez szükséges elmélet a [linken](#) található dokumentumban olvasható, jegyzőkönyvem ezt dolgozza fel.

4. Mérési feladatok

A mérés során elkészítendő szimulációs program a 6. fejezetben olvasható szimulációs program segítségével készült. Maguk a feladatok a következők voltak.

4.1. Conway-féle életjátékot szimuláló sejtautomata

Ebben a szakaszban több szimulációt is készítettem, melyek mindegyikét meg is animáltam, és összegyűjtöttem egy listába, mely itt látható:

1. Sejtautomata rendszer szimulációja 20 másodpercen keresztül $n = 1$ szabállyal, 0.5-ös kezdeti élő sejtsűrűséggel, és periódikus határfeltétellel. ([link](#))

2. Sejtautomata rendszer szimulációja 200 másodpercen keresztül $n = 2$ szabállyal, 0.99-es kezdeti élő sejtsűrűséggel, és élő határfeltétellel (ezzel minden meghal az első pillanatban, és egy négyzetes hullám indul befelé középre). ([link](#))
3. Sejtautomata rendszer szimulációja 100 másodpercen keresztül $n = 3$ szabállyal, 0.9-es kezdeti élő sejtsűrűséggel, és élő határfeltétellel. ([link](#))
4. Sejtautomata rendszer szimulációja 20 másodpercen keresztül $n = 7$ szabállyal, 0.9-es kezdeti élő sejtsűrűséggel, és periódikus határfeltétellel. ([link](#))

Ennél több szimuláció megtekinthető a feladatmappámban, de ezek a legérdekesebbek, ezek közül is talán a másodikat emelném ki. n életfeltételt meghatározó szám emelése 2 fölött nem igazán eredményez élhető körülményeket a sejtautomatáknak, jellemzően hamar inaktívvá válik a rendszer, 1, és 2 értékek mellett jellemzi csak a pezsgő élet.

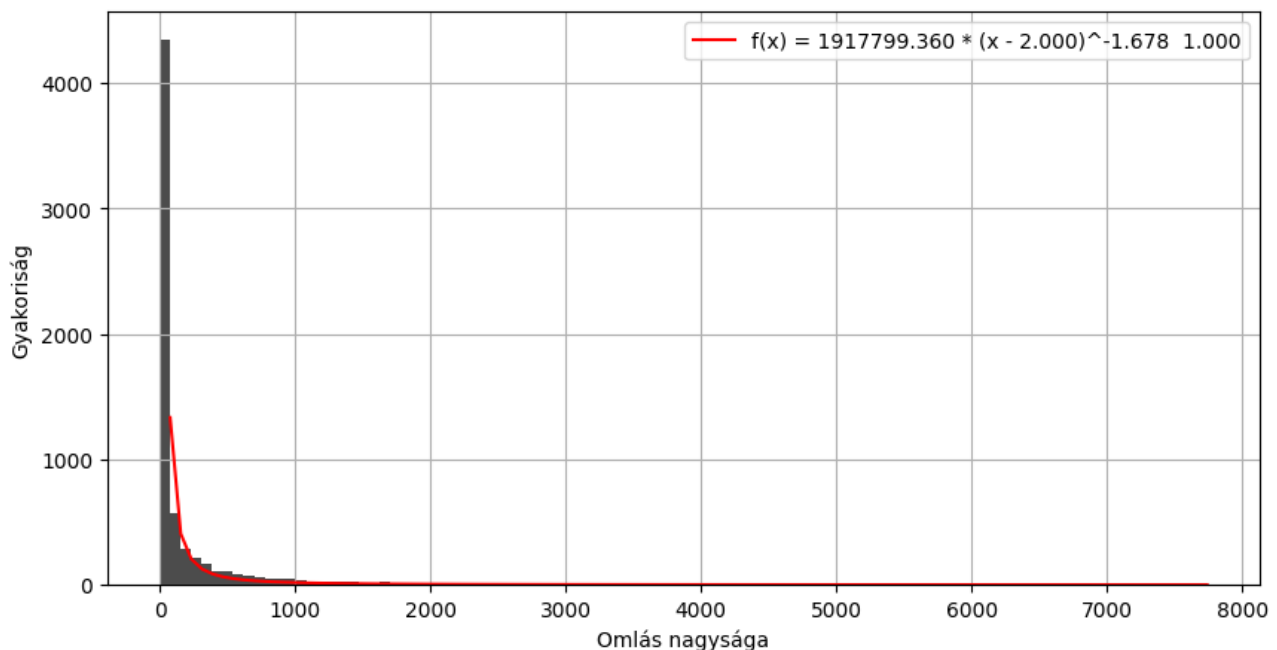
4.2. 2D Homokdomb modell skálázási szabálya

Ehhez a modellhez is készítettem egy [szimulációt](#), mely erdőtűz-szerű színezést kapott.

A program segítségével meghatároztam, hogy milyen tarfolt/omlás eloszlás jellemzi a szimulációt, amikor $n = 4$ értékre állítom az omlás kritikus értékét. A 4.1. ábrán látható eloszlásra illesztett függvény a következő alakú:

$$f(x) = a(x - d)^c + b \quad (4.1)$$

Ezzel a függvénnyel több illesztést is végeztem, hogy meghatározzam a $P(s) = s^{-\tau}$ -vel jellemezhető homokdomb modell skálázási együtthatóját τ -t (s az omlás nagysága). Azt tapasztaltam, hogy ez a képen látható érték esetében $\tau = 1.678$ -nak is adódhat, de adódhat ennél jelentősebben kevesebbnek, attól függően, a rendszert mekkorra kezdeti futás elhanyagolásával, milyen paraméter illesztési határokat alkalmazok, illetve az is, hogy mekkora rácson szimulálok a modellt. Jellemzően nekem inkább 1.-1.1 körüli értékek adódtak, míg az irodalom szerint erre inkább 1.5 körüli és annál nagyobbak várhatóak.



4.1. ábra. Az omlások nagyságának valószínűség eloszlása, és a rá illesztett hiperbólikus függvény.

5. Diszkusszió

A vizsgálat során a sejtautomata-modellek viselkedését elemeztük különféle kezdeti konfigurációk, iterációk és szabályrendszerek mentén. A szimulációs eredmények alapján jól megfigyelhető volt, hogy az automata dinamikája nagymértékben függ a kezdeti állapotoktól és a szabályok jellegétől.

A determinisztikus szabályokkal (pl. Conway-féle életjáték) szimulált rendszerek esetén különösen érdekes struktúrák alakultak ki. Például a „glider” típusú alakzatok mozgása, valamint a stabil és oszcilláló mintázatok megjelenése jól példázza, hogy egyszerű szabályokból is komplex viselkedés alakulhat ki. Az ilyen rendszerekre jellemző az úgynevezett emergens viselkedés, ahol a lokális interakciókból globálisan rendezett minták születnek.

A homokdomb-modell esetében a rendszer evolúciója az önszerveződő kritikusság (self-organized criticality, SOC) jelenségét mutatta. A domb fokozatos feltöltésével elértük azt a küszöbértéket, ahol az „összeomlások” (avalanches) egyensúlyi állapothoz vezettek. A modell jól szemlélteti, hogyan képes egy természetes rendszer finomhangolás nélkül kritikus állapotban stabilizálódni.

Mindkét modell alátámasztja, hogy sejtautomaták alkalmasak komplex fizikai, biológiai és társadalmi jelenségek szimulációjára és kvalitatív vizsgálatára. Az életjáték például hasznos lehet biológiai populációk vagy vírusok terjedésének modellezésében, míg a homokdomb-modell földrengések, gazdasági válságok vagy más láncreakciószerű jelenségek megértéséhez nyújthat eszközt.

A szimulációk során felmerült legfőbb korlát az iterációk számának növekedésével járó számítási költség, illetve a determinisztikus szabályokból adódó kiszámíthatóság korlátai voltak. A jövőbeni munkák során a modellbe véletlenszerűség (stochaszticitás) beépítése, illetve többdimenziós rendszerek vizsgálata révén a rendszer még gazdagabb viselkedése lenne feltárható.

6. Szimulációs program (simulator.cpp)

```

1  #include <iostream>
2  #include <vector>
3  #include <fstream>
4  #include <cstdlib>
5  #include <ctime>
6
7  using namespace std;
8
9  int n_rows, n_cols, n_rule, run_time;
10 double density;
11 string file_name;
12
13 enum BoundaryCondition { OPEN, PERIODIC, ALIVE, RANDOM };
14
15 BoundaryCondition boundary;
16
17 string simulation_type = "GameOfLife";
18
19 int seed = time(0);
20
21 struct GameOfLife {
22     int rows, cols, n_rule;
23     vector<vector<int>> grid, newGrid;
24     BoundaryCondition boundary;
25
26     GameOfLife(int r, int c, int n, BoundaryCondition b)
27         : rows(r), cols(c), n_rule(n), boundary(b),
28           grid(r, vector<int>(c, 0)), newGrid(r, vector<int>(c, 0)) {
29         srand(seed);
30     }
31
32     void randomInitialize(double density) {
33         for (int i = 0; i < rows; ++i)
34             for (int j = 0; j < cols; ++j)
35                 grid[i][j] = ((double)rand() / RAND_MAX < density) ? 1 : 0;
36     }
37
38     int getCell(int i, int j, double density) {
39         switch (boundary) {
40             case OPEN:
41                 if (i < 0 || i >= rows || j < 0 || j >= cols)
42                     return 0;

```

```

43         return grid[i][j];
44     case PERIODIC:
45         if (i < 0) i = rows - 1;
46         else if (i >= rows) i = 0;
47         if (j < 0) j = cols - 1;
48         else if (j >= cols) j = 0;
49         return grid[i][j];
50     case ALIVE:
51         if (i < 0 || i >= rows || j < 0 || j >= cols)
52             return 1;
53         return grid[i][j];
54     case RANDOM:
55         if (i < 0 || i >= rows || j < 0 || j >= cols)
56             return ((double)rand() / RAND_MAX < density) ? 1 : 0;
57         return grid[i][j];
58     }
59     return 0;
60 }
61
62 void step(double density) {
63     for (int i = 0; i < rows; ++i)
64         for (int j = 0; j < cols; ++j) {
65             int liveNeighbors = 0;
66             for (int dx = -1; dx <= 1; ++dx)
67                 for (int dy = -1; dy <= 1; ++dy)
68                     if (dx != 0 || dy != 0)
69                         liveNeighbors += getCell(i + dx, j + dy, density);
70
71             if (liveNeighbors == n_rule + 1)
72                 newGrid[i][j] = 1;
73             else if (liveNeighbors == n_rule)
74                 newGrid[i][j] = grid[i][j];
75             else
76                 newGrid[i][j] = 0;
77         }
78     grid = newGrid;
79 }
80
81 void display(ofstream& file) {
82     for (int i = 0; i < rows; ++i) {
83         for (int j = 0; j < cols; ++j) {
84             if (j == cols - 1) file << grid[i][j];
85             else file << grid[i][j] << "\t";
86         }
87         file << "\n";
88     }
89 }
90
91 void run(int generations, ofstream& file, double density) {
92     for (int gen = 0; gen < generations; ++gen) {
93         display(file);
94         step(density);
95     }
96 }
97 };
98
99 struct SandDune {
100     int rows, cols, n_rule;
101     vector<vector<int>> grid, newGrid;
102     int slilage_count = 0;
103     ofstream display_file;
104     ofstream slilage_file;
105

```

```

106 SandDune(int r, int c, int n, string file_name)
107     : rows(r), cols(c), n_rule(n), display_file(file_name + ".txt"), slipage_file(file_name + "_slipage.txt"),
108     grid(r, vector<int>(c, 0)) {srand(seed);}
109
110 void Initialize() {
111     for (int i = 0; i < rows; ++i)
112         for (int j = 0; j < cols; ++j)
113             grid[i][j] = 0;
114 }
115
116 void step(int time) {
117     int randomRow = rand() % rows;
118     int randomCol = rand() % cols;
119     grid[randomRow][randomCol] += 1;
120     if (grid[randomRow][randomCol] == n_rule) induceSlipage(randomRow, randomCol);
121     for (int i = 0; i < rows; ++i)
122         for (int j = 0; j < cols; ++j) {
123             if (grid[i][j] > n_rule) {
124                 slipage_file << "Fatal error: cell (" << i << ", " << j << ") has more than " << n_rule << " grains of sand\n";
125                 exit(1);
126             }
127         }
128     slipage_file << time << "\t" << slipage_count << '\n';
129     slipage_count = 0;
130 }
131
132 void induceSlipage(int i, int j) {
133     display();
134     grid[i][j] = 0;
135     slipage_count++;
136     int dx[4] = {1, -1, 0, 0};
137     int dy[4] = {0, 0, 1, -1};
138     for (int k = 0; k < 4; ++k) {
139         int ni = i + dx[k];
140         int nj = j + dy[k];
141         if (ni >= 0 && ni < rows && nj >= 0 && nj < cols) {
142             grid[ni][nj] += 1;
143             if (grid[ni][nj] == n_rule) induceSlipage(ni, nj);
144         }
145     }
146 }
147
148
149 void display() {
150     for (int i = 0; i < rows; ++i) {
151         for (int j = 0; j < cols; ++j) {
152             if (j == cols - 1) display_file << grid[i][j];
153             else display_file << grid[i][j] << "\t";
154         }
155         display_file << '\n';
156     }
157 }
158
159 void run(int time) {
160     for (int t = 0; t < time; ++t) {
161         step(t);
162         display();
163     }
164 }
165 };
166
167 void getInput (int argc, char** argv) {
168

```

```

169     if (argc != 8 && argc != 6) {
170         cout << " Usage 1: " << argv[0] << " n_rows n_cols n_rule boundary_condition density run_time file_name" << endl;
171         cout << " n_rows: number of rows in the grid" << endl;
172         cout << " n_cols: number of columns in the grid" << endl;
173         cout << " n_rule: number of live neighbors to keep a cell alive" << endl;
174         cout << " boundary_condition: OPEN, PERIODIC, ALIVE, RANDOM" << endl;
175         cout << " density: initial density of live cells (0.0 to 1.0)" << endl;
176         cout << " run_time: number of generations to run" << endl;
177         cout << " file_name: name of the output file" << endl;
178         cout << " This program simulates the Game of Life." << endl;
179         cout << " Usage 2: " << argv[0] << " n_rows n_cols n_rule run_time file_name" << endl;
180         cout << " n_rows: number of rows in the grid" << endl;
181         cout << " n_cols: number of columns in the grid" << endl;
182         cout << " n_rule: number of live neighbors to keep a cell alive" << endl;
183         cout << " file_name: identifying name of the output file" << endl;
184         cout << " This program simulates the Sand Dune model." << endl;
185         exit(1);
186     }
187
188     if (argc == 6) {
189         n_rows = atoi(argv[1]);
190         n_cols = atoi(argv[2]);
191         n_rule = atoi(argv[3]);
192         run_time = atoi(argv[4]);
193         file_name = argv[5];
194         simulation_type = "SandDune";
195         if (n_rule < 1) {
196             cout << "n_rule must be greater than 0." << endl;
197             exit(1);
198         }
199         return;
200     }
201     else if (argc == 8) {
202         n_rows = atoi(argv[1]);
203         n_cols = atoi(argv[2]);
204         n_rule = atoi(argv[3]);
205         string boundary_condition = argv[4];
206         density = atof(argv[5]);
207         if (boundary_condition == "OPEN") {
208             boundary = OPEN;
209         } else if (boundary_condition == "PERIODIC") {
210             boundary = PERIODIC;
211         } else if (boundary_condition == "ALIVE") {
212             boundary = ALIVE;
213         } else if (boundary_condition == "RANDOM") {
214             boundary = RANDOM;
215         } else {
216             cout << "Invalid boundary condition. Use OPEN, PERIODIC, ALIVE, or RANDOM." << endl;
217             exit(1);
218         }
219         run_time = atoi(argv[6]);
220         file_name = argv[7];
221         if (n_rule < 1) {
222             cout << "n_rule must be greater than 0." << endl;
223             exit(1);
224         }
225     }
226 }
227
228 int main(int argc, char** argv) {
229
230     getInput(argc, argv);
231

```

```
232     if (simulation_type == "SandDune") {
233         SandDune sand_dune(n_rows, n_cols, n_rule, file_name);
234         sand_dune.Initialize();
235         sand_dune.run(run_time);
236         return 0;
237     }
238     else if (simulation_type == "GameOfLife") {
239         ofstream file(file_name);
240
241         GameOfLife game(n_rows, n_cols, n_rule, boundary);
242         game.randomInitialize(density);
243         game.display(file);
244         game.run(run_time-1, file, density); // run_time-1 because we already displayed the initial state
245         file.close();
246     }
247
248
249     return 0;
250 }
```